7. Structs, Methods, and Pointers

07. Structs, Methods, and Pointers

Phase: Phase 1 — CS fundamentals through Go
Estimated time: 1–2 days
Slug: `07-structs-methods-pointers`

Learning objectives

- Define structs and compose them with embedding
- Choose value vs pointer receivers deliberately
- Explain what a pointer stores (an address)
- Avoid nil pointer dereferences

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Structs

```
type User struct {
    Name string
    Age  int
}

u := User{Name: "Ada", Age: 36}
```

Pointers

A pointer holds a **memory address** of a value.

```
p := &u       // *User
p.Age = 37    // Go auto-dereferences
```

Use pointers when: - you need to mutate the caller's struct - the struct is large and you want to avoid copies - you need `nil` to mean "absent"

Methods

```
func (u User) Greet() string { return "hi " + u.Name }
func (u *User) Birthday() { u.Age++ }
```

Rule of thumb: if any method needs a pointer receiver, be consistent for the type. Value receivers get a copy — mutations won't stick.

Common mistakes (especially from Python instincts)

- Expecting Python-like always-reference objects — Go structs are values by default
- Mixing value and pointer receivers inconsistently
- Returning pointers to loop variables incorrectly (fixed in modern Go, still understand escapes)

Exercises

Work in `exercises/07-structs-methods-pointers/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/07-structs-methods-pointers/` until you have an honest failing attempt.

Self-check

1. When should a method use a pointer receiver?
2. What does `&` and `*` mean?
3. What happens if you call a method on a nil pointer with a pointer receiver?

Further reading

- https://go.dev/tour/moretypes
- https://go.dev/doc/effective_go#pointers_vs_values